Toronto Metropolitan University
Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

BV06 - Intelligence-Based Load Forecasting, Hosted on Cloud

by

Rayyan Khan, Jason Chen, Jason Fu, and Adam Roth

Final EDP Report

Electrical Engineering Capstone Design Project

Toronto Metropolitan University, 2026

Acknowledgements

The project team would like to acknowledge the assistance provided by Dr. Bala Venkatesh and Shima BZ Homayie, Ph.D., whose guidance informed each step of the engineering design process.

Certification of Authorship

By signing this document, I hereby certify that the works presented within this report are my own and adhere to Policy 60: Academic Integrity at Toronto Metropolitan University.



Name: Rayyan Khan
Date: 4/10/2026



Name: Jason Chen
Date: 4/10/2026



Name: Jason Fu
Date: 4/10/2026



Name: Adam Roth
Date: 4/10/2026

Table of Contents

Acknowledgements	2
Certification of Authorship	3
Abstract	5
Introduction & Background	6
Objectives	8
Theory and Design	9
Material/Components List	17
Measurement and Testing Procedures	18
Performance Measurement Results	19
Analysis of Performance	22
Conclusions	23
References	24

Abstract

Load forecasting is the process of predicting electricity demand given factors such as weather conditions. Effective load forecasting can result in less energy wastage and allow opportunities for more environmentally conscious forms of energy production to be utilised within the grid.

The objective of this engineering design project is to design and implement an intelligence-based load forecasting system, hosted on the cloud. The project consists of four key components: a forecasted weather sourcing module, an intelligence model for industrial and commercial load power forecasting, an intelligence model for residential load power forecasting, and a cloud platform to host the modules and allow user access via a web interface.

The data collection component consists of a historical data collection from 2020-2024 on Toronto, Ontario, Alberta and British Columbia weather conditions (temperature, relative humidity, and dew point) as well as residential, commercial, and industrial energy consumption (in MWh) in usable spreadsheet formats for model training. The forecasted weather sourcing has been developed in Python using API connections to OpenWeatherMap and OpenMeteo, where realtime and previous 72/168-hours of information on the same weather conditions mentioned previously can be obtained for producing load forecasts.

Two intelligence models have been developed in Python for load forecasting, the Convolutional Neural Network (CNN) model and the Long Short Term Memory (LSTM) model. The CNN model reads CSV data on weather and energy consumption, then utilises layer architecture to produce a 24-hour load forecast. The LSTM model similarly collects CSV datasets and uses time series prediction to output 72-hour load forecasts based on the previous 48 hours of data. Both models achieved a nMAPE of below or near ~5%, well within the acceptable range for intelligence models of these types.

A web interface using Azure Cloud has been developed, one that allows users to access load forecasts for four pre-determined locations, filtered by sector, and then download the results as a CSV file. The website also allows users to train their own forecasting model by downloading a data file template and uploading their own data. After training, the model performance is presented graphically and evaluated, and the resulting keras file is available for download.

Introduction

Load forecasting is the process of predicting electricity demand on the power grid using information from various relevant factors. Load forecasts can be short-term (hourly, daily) or long-term (months, years), with corresponding data to match the type of forecast. Long-term forecasts are typically performed using information on climate change, economic trends, and energy policy changes, whereas short-term forecasts rely more closely on current weather information and seasonal changes [1]. By understanding energy consumption trends and the predicted demand, effective forecasting can help reduce energy wastage and inefficiency, while also allowing for proper allocation of renewable energy sources that are typically condition-dependent [1].

The advancement of technologies such as artificial intelligence and machine learning have allowed for the process of load forecasting to grow more efficient, thanks to the capabilities of these models being able to process larger amounts of data and form stronger correlations between datasets, ultimately leading to increased accuracy [1].

The purpose of this electrical engineering design project then is to train and develop an intelligence-based load forecasting module that is hosted on the cloud. Using historical data on weather, as well as on residential, commercial and industrial energy consumption, an intelligent model is to be trained in order to accurately predict energy demand on the grid. The forecasting module is then to be hosted on Azure cloud, where users can interact with a graphical user interface in order to access the model and obtain load forecasts. The project is structured as a four-member team effort, where each member is responsible for a specific module that contributes to the overall system.

The first component of the project is historical data collection and the forecasted weather sourcing module, which combine historical data on weather and energy consumption along with realtime weather information to aid the intelligence models in forecasting predictions. This module ensures that both historical data and real-time weather information are available and properly formatted for use in the forecasting models.

The second module is the residential forecasting model, which utilizes a Long Short-Term Memory (LSTM) network to learn patterns in electricity consumption using the historical data provided by the data sourcing module. The trained model then uses live weather data from the API to generate a 24-hour forecast of residential electricity demand.

The third module focuses on commercial and industrial forecasting using a Convolutional Neural Network (CNN) model. Similar to the residential model, it is trained on historical data and uses real-time weather inputs to predict future electricity consumption for a 24 hour window as well.

Finally, all modules are integrated through Microsoft Azure using its app service functionality, allowing the system to operate in a cloud environment. This setup enables communication between modules and supports uploading and managing files through GitHub as a centralized workspace for collaboration and deployment.

Background

This project focuses on short-term electrical load forecasting, which is used to predict future energy demand over a 24-hour period. Accurate load forecasting is essential for maintaining the grid as overgeneration can cause grid instability leading to issues such as blackouts [2].

Residential electrical load is influenced by several factors, including weather conditions such as temperature and humidity, as well as time based patterns like hourly, daily, and seasonal variations. To better represent these effects, features such as Heating Degree Days (HDD), Cooling Degree Days (CDD), ‘isWeekend’ and cyclical time components are used to allow the model to learn these binary-like patterns better. Residential trends tend to follow weather conditions, as temperature changes directly influence heating and cooling demand.

Commercial and business electrical load is influenced by mostly the same factors as residential load. These features include temperature, humidity, as well as time-based patterns such as daily and weekly operating schedules. However some features such as HDD and CDD are not relevant in this case as these features more accurately describe residential trends rather than commercial and industrial trends. Commercial and industrial trends tend to follow a more structured pattern, as work hours, occupancy schedules, and holidays can significantly affect the overall electricity demand during specific time frames.

Objectives

The objective of this engineering design project is to design and implement an intelligence-based load forecasting system, hosted on the cloud. The data sourcing module focuses on collecting historical load data and integrating real-time weather data through an API. The residential forecasting module uses an LSTM model to predict residential electricity demand, while the commercial and industrial module uses a CNN model to forecast load for those sectors. The final module focuses on Azure integration, ensuring that all components are connected, allowing data to flow between modules and enabling deployment through a centralized cloud platform.

The objective of the data sourcing module is to collect and prepare the datasets required for training the forecasting models. This includes collecting historical electricity consumption data from system operators and weather data for the same time period, including features such as temperature, humidity, and dew point. The other objective for this component is to utilise live weather APIs that allow real-time and previous 168/72-hour forecast data to be pulled as input for the models. This ensures that the trained forecasting models can use current weather conditions during deployment, allowing them to generate updated 24-hour load forecasts based on live environmental data.

The objective of the LSTM module is to train and tune models based on the historical data provided by the data sourcing module. The primary goal is to develop a model for the Toronto region that achieves around ~5% MAPE as a metric. This involves optimizing the model architecture and feature set to accurately capture residential load patterns and produce reliable 24-hour forecasts. Once this baseline is achieved, the focus is extended to secondary regions, including the Alberta and British Columbia provinces, to evaluate the model's performance across different datasets and ensure its generalizability.

The objective of the CNN module is to train and tune models based on the historical data provided by the data sourcing module. The primary goal is to develop a model for the Toronto region that achieves strong performance based on metrics such as nMAPE, R^2, RSE, and MASE. This involves optimizing the model architecture and feature set to accurately capture commercial and industrial load patterns and produce reliable 24-hour forecasts. Once this baseline is achieved, the focus is extended to secondary regions, including the provinces of Alberta and British Columbia, to evaluate the model's performance across different datasets and ensure its generalizability.

Lastly, the objective of the Azure module is to combine and integrate all these modules together within a cloud based environment. This involves enabling communication between the data sourcing, forecasting, and deployment components, allowing data to flow across the system. The Azure platform is used to host the services, manage the workflow, and support deployment, ensuring that the overall system operates reliably.

Theory and Design

Historical Data Collection

In order to perform load forecasting, the intelligence models required historical data on weather and energy consumption to be trained on. The selected time interval for historical data collection was chosen to be 2020-2024, as this allowed for five full years of usable data that also reflected current weather and energy consumption patterns. Seeing as 2025 was not complete at the time of initial project development, 2025 was left out of data collection and instead used as validation once the year had completed. All collected datasets were compiled into easy to read spreadsheets for model training.

Hourly historical energy consumption information was collected from regional system operators, including the IESO for Ontario, AESO for Alberta, and BCHydro for B.C.. These regions were selected in order to ensure the intelligence models could work for a variety of weather conditions and energy consumption patterns. Energy consumption data was divided by sector, including residential, commercial and industrial, and the total consumption was recorded in MWh.

The Environment Canada historical weather database was used for hourly historical weather data collection for Toronto, Ontario, Alberta and British Columbia. Weather information collected included temperature (in ^{o}C), dew point, and humidity percentage. Toronto being the only city in the region lineup was due to the lack of specific energy consumption data for cities in Alberta and B.C., thus the weather data was collected at the provincial level as well.

Forecasted Weather Sourcing Module

The forecasted weather sourcing module was written as a callable function within a Python script. The program uses an API connection to OpenWeatherMap, a free-to-use weather API, in order to collect real-time weather data on temperature (in ^{o}C), humidity percentage, precipitation (in mm), wind speed (m/s) and pressure (hPa). For 72/168-hour previous weather forecast information, the module utilises an API connection to OpenMeteo, where the same weather data is acquired. Both sets of data are collected in Pandas dataframe tables and CSV files, where the weather condition values can then be used in other modules.

Initially, the MSC GeoMet API provided by Environment Canada was planned to be used for the module, however the difficult-to-access endpoints and overall complications with the Environment Canada services led to the need for a different API. OpenWeatherMap was then selected for its ease of use and robust features, such as 4-day hourly forecast. However, OpenWeatherMap did not allow for historical data access, even on a student plan. Thus, OpenMeteo was consolidated into the script to allow for free, hourly historical weather data to be collected for intelligence model usage.

Residential Forecasting

Model Chosen: LSTM

Reasoning:

This model was chosen to forecast residential electricity consumption as the LSTM model uses previous patterns to forecast future outputs, and unlike some models, LSTM is designed to learn long term trends while being able to react to short term trends [3]. This is especially useful for this design since weather data are used as inputs to the model. Weather data contain long term yearly trends due to seasonal changes, as well as short term daily trends caused by the day and night cycle. This is especially useful because weather patterns are closely tied to electricity consumption, as heating, cooling, and lighting demands vary with both season and time of day.

The training data will be split with 60% training, 20% validation and 20% testing to ensure accuracy. The model will be trained for 200–250 epochs to avoid over fitting, given that the current training dataset contains approximately 47,000 entries [4]. The model will take in an input of 48 Hour window of weather data to output the next 24 Hours of electricity consumption as forecast.

Model Specifications:

The forecasting model was built using a sequential LSTM architecture. It contains one 64 unit bidirectional LSTM layer followed by a dropout of 0.2 to prevent overfitting, then a 32-unit unidirectional LSTM layer. A 64 dense layer with ReLU activation is used before the final output layer. The output layer uses a linear activation function to predict the 24-hour forecast horizon. The model was trained using mean squared error as the loss function, Adam optimizer with a learning rate of 0.0005, and root mean squared error as the evaluation metric. Early stopping with a limit of 15 epochs was used to reduce overfitting.

Features:

Features such as temperature, time (hours), dew point, and humidity were used as input features, as they are strongly correlated with electricity demand and were consistently available in the dataset. Although precipitation was also available, it was excluded due limited variation, as values were frequently zero, reducing its impact on model performance .

Additional features such as HDD, CDD, and ‘isWeekend’ are also used to better represent the factors that influence residential electricity consumption. HDD and CDD capture the impact of temperature on heating and cooling demand, which are major drivers of residential load. The ‘isWeekend’ feature helps the model account for differences in daily routines, as energy usage patterns typically change between weekdays and weekends. Together, these features allow the model to better learn variations in consumption and improve forecasting accuracy.

Cyclical time features are used to help the model understand repeating patterns in time, such as hourly, daily, and seasonal cycles. Since electricity demand follows periodic trends, representing

time in a cyclical form allows the model to better capture these patterns. This improves the model's ability to learn consistent variations in load and produce more accurate forecasts.

nMAPE:

The Normalized Mean Absolute Percentage Error equation is presented in Eq.(1) below. nMAPE measures the error by taking the sum of all absolute errors over the sum of all values. This method had to be implemented over the normal MAPE method as the normalized data resulted in the model flatlining due to the scale difference of temperature and electricity consumption data.

$$nMAPE\ (\%) = \frac{\sum_{i=1}^{n} |real(i) - npred(i)]}{[real(i)]} \cdot 100 \quad (1)$$

RMSE:

Root Mean Square Root Error measures the average of squared errors then takes the square root of it. The RMSE equation is showcased in Eq.(2) below.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (real(i) - pred(i))^2} \quad (2)$$

Commercial / Industrial Forecasting

Model Chosen: CNN

This model implements a deep learning-based electrical load forecasting system using a Convolutional Neural Network (CNN). A Convolutional Neural Network (CNN) was selected for the intelligent load forecasting model due to its effectiveness in extracting meaningful patterns from multivariate time-series data. By applying one-dimensional convolutional filters over historical weather inputs, the CNN is able to capture short-term temporal dependencies and recurring daily trends that strongly influence electrical demand. Unlike recurrent models such as LSTMs, CNNs offer faster training, improved computational efficiency, and greater stability, while still effectively learning relationships between multiple input features. This makes the CNN well-suited for mapping past weather conditions to future load demand, particularly in a deployment setting where predictions are generated using weather data alone without relying on previous load values. The primary objective of the model is to predict hourly power consumption for the next 24 hours of Commercial and Industrial sectors using only weather data and time-based features, without relying on historical load as an input during inference. This design enables a fully deployable and scalable forecasting system that can operate in real time using publicly available weather data. The overall system consists of two main components: a training pipeline, which learns the relationship between environmental conditions and electrical demand, and a forecasting pipeline, which uses live weather data to generate future load predictions.

The model is trained on a merged dataset (previously two separate CSVs) containing historical power consumption data alongside corresponding weather variables, including temperature, relative humidity, and dew point temperature. In addition to these raw inputs, the timestamp associated with each data point is used to derive a set of engineered features that capture temporal patterns in electricity usage. These include hour of the day, day of the week, month of the year, and a binary indicator for weekends. To properly represent the cyclical nature of time, sine and cosine transformations are applied to the hour, day, and month features. This encoding ensures continuity in the feature space, allowing the model to understand that values such as 23:00 and 00:00 are temporally close. Furthermore, heating degree days (HDD) and cooling degree days (CDD) are calculated using a base temperature of 18°C. These features quantify the extent to which temperatures deviate from a comfort baseline and are particularly useful for capturing load variations driven by heating and cooling demand.

As discussed earlier with the LSTM model, a correlation matrix was generated based on the weather data in correlation to power consumption in regards to determining which feature impacts the forecast and to what degree. These parameters were chosen based on the correlation matrix illustrated in **Figure 1** below.

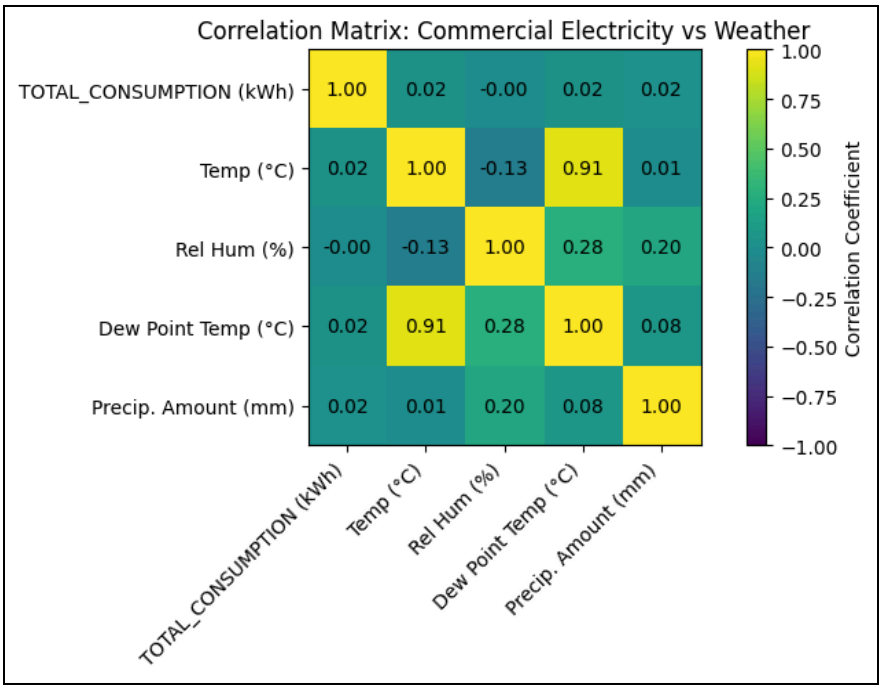


Figure 1. CNN Correlation Matrix

The dataset is first cleaned by removing missing or invalid entries, converting all relevant columns to numeric types, and ensuring chronological ordering. Duplicate timestamps are eliminated to maintain data integrity. The processed dataset is then transformed into a supervised learning format using a sliding window approach. Each training sample consists of two inputs and one output: the first input is a sequence of weather and time features over the past 72 hours, the second input is a sequence of forecasted weather and time features for the next 24 hours, and the output is the corresponding electrical load over the next 24 hours. This formulation creates a dual-input sequence-to-sequence learning problem that allows the model to simultaneously consider historical trends and future environmental conditions when making predictions.

The core of the model is a dual-branch CNN architecture designed to process past and future inputs separately before combining their learned representations. The first branch processes the past 72-hour input using a series of one-dimensional convolutional layers with causal padding, ensuring that temporal ordering is preserved. Dilated convolutions are also employed in this branch to expand the receptive field, allowing the model to capture longer-term dependencies without significantly increasing computational complexity. Batch normalization and dropout layers are included to stabilize training and reduce overfitting. The output of this branch is then reduced using global max pooling to extract the most salient temporal features.

The second branch processes the 24-hour future weather input using standard convolutional layers. Unlike the past branch, causal padding is not required since the future data is already

aligned with the prediction horizon. This branch focuses on extracting patterns from expected future conditions that may influence load demand, such as anticipated temperature changes. Similar to the first branch, batch normalization and dropout are applied, followed by global max pooling to produce a compact feature representation.

The outputs from both branches are concatenated and passed through a series of fully connected layers. These dense layers further refine the combined feature space and enable complex nonlinear relationships to be learned. Dropout is again applied at this stage to improve generalization. The final output layer consists of 24 neurons with linear activation, each corresponding to a predicted load value for one hour in the forecast horizon. This architecture allows the model to produce a full 24-hour forecast in a single forward pass.

During training, the dataset is split into training, validation, and testing subsets using a 70-15-15 ratio while preserving temporal order to avoid data leakage. Feature scaling is performed using a standardization approach, where both input features and target values are normalized based on statistics computed from the training set. This ensures that all variables contribute equally to the learning process and improves convergence stability. The model is trained using the Adam optimizer with a learning rate of 0.001, and mean squared error (MSE) is used as the loss function. Additional training strategies, including early stopping and learning rate reduction on plateau, are implemented to prevent overfitting and ensure efficient convergence.

Model performance is evaluated using a comprehensive set of metrics, including mean absolute error (MAE), root mean squared error (RMSE), mean absolute percentage error (MAPE), symmetric MAPE (sMAPE), coefficient of determination (R^2), normalized RMSE (NRMSE), and mean absolute scaled error (MASE). These metrics provide a well-rounded assessment of both absolute and relative prediction accuracy. An approximate forecasting accuracy is also defined as 100 minus the MAPE value, providing an intuitive measure of performance. These metrics are calculated by the model and printed, with the result shown in **Figure 2** below.

```
===== FINAL TEST PERFORMANCE =====
MAE: 7319.4214
MSE: 118116656.0000
RMSE: 10868.1487
MAPE_percent: 5.8422
sMAPE_percent: 5.6077
R2: 0.8613
NRMSE: 0.0917
MASE: 1.1165
Approx_Forecast_Accuracy_percent: 94.1578
```

Figure 2. Printed CNN Performance Metrics

The training model also prints a visual representation of the model accuracy by graphing the predicted values it creates based on learned patterns against actual values acquired from the historical data it is being trained on. The result is showcased in **Figure 3** below.

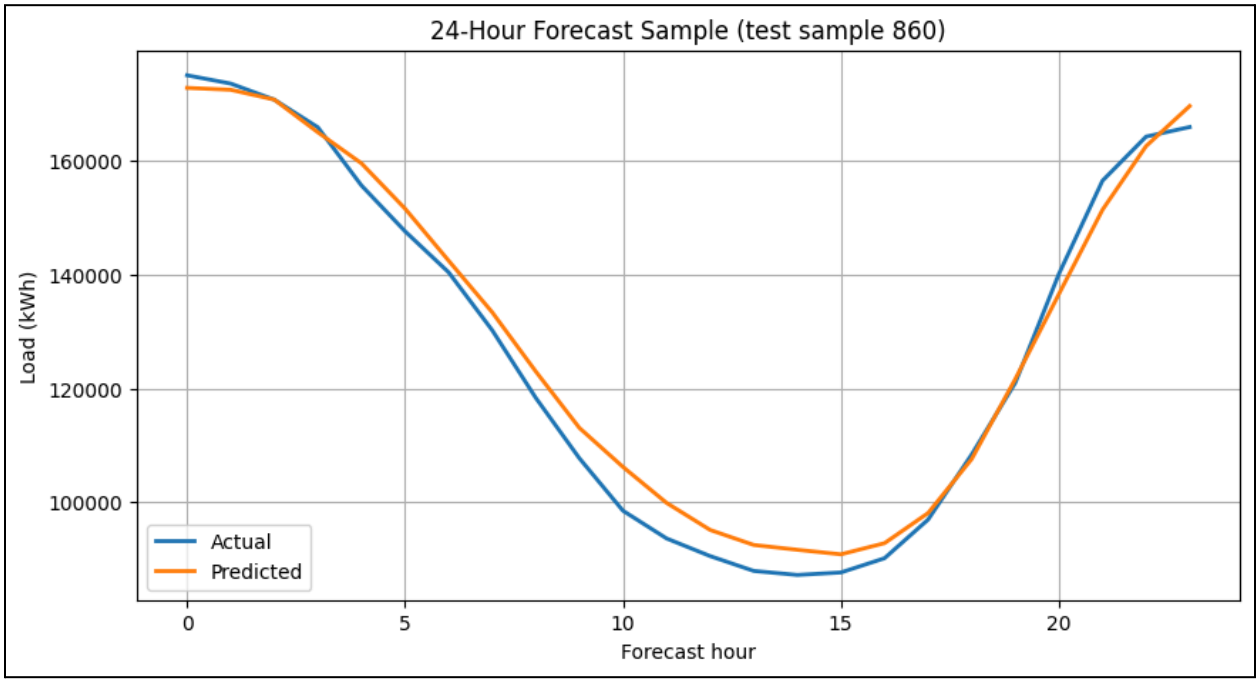


Figure 3. Plot of CNN Forecast vs Actuals

Once trained, the model and its associated preprocessing components, including feature and target scalers, are saved as artifacts for deployment. The forecasting pipeline utilizes these artifacts to generate predictions in real time. Weather data is retrieved from the Open-Meteo API, which provides both historical and forecasted weather information. Specifically, the system fetches the past 72 hours of weather data and the next 24 hours of forecasted conditions. These inputs are then processed using the same feature engineering steps applied during training to ensure consistency. The resulting data is reshaped and scaled before being passed into the trained CNN model.

The model outputs a 24-hour load forecast, which is then inverse-transformed back to the original scale. The final results are presented in a structured format that includes timestamps, predicted load in megawatts for easier interpretation. A key design decision in this project is the exclusion of historical load as an input during inference. Instead, the model relies solely on weather and temporal features, having learned the underlying relationships during training. This significantly simplifies deployment, as it removes the need for real-time access to utility load data while still maintaining strong predictive performance. This forecast model is stored on Github which gets accessed by Azure to display the final 24 hour forecasts requested by the front end user.

Website Design

The website functions as the user interface and the main interaction layer of the system. The primary role of the website frontend is to collect user input (uploaded files), package the data into a structured JSON format, and then send it to the backend for processing using an HTTPS request to a server-side endpoint. The backend framework is programmed in HTML, and after the backend returns a response from the ML models, the website displays the load forecast output in a user-friendly graphical and tabular format.

Regarding hosting, the website is deployed on Azure using a standard cloud deployment approach, where the web application is hosted as an Azure web service, and the sourcing module and intelligence model source codes are hosted on a GitHub repository. The Azure web service was then configured to deploy directly from this repository, meaning any updates to the main branch can be automatically pulled and published to the live website.

The website allows users to choose between four different locations for load forecasting, including the city of Toronto, as well as the provinces of Ontario, Alberta and British Columbia. Using a dropdown menu, users can choose between residential, industrial or commercial forecasting, and the model test metrics are displayed in a separate box below once a model has been selected. Through another dropdown box, website users can then choose whether to utilise the forecasted weather sourcing module and its API connections to conduct a realtime weather forecast for the next 24-hours, or instead they can opt to train their own model by downloading a data CSV template and uploading their own data using the template's format. The results of the forecast or model training are then displayed in both a graph and a table on the right side of the website. **Figure 4** below displays the website's user interface when the residential forecasting model has been selected for Toronto.

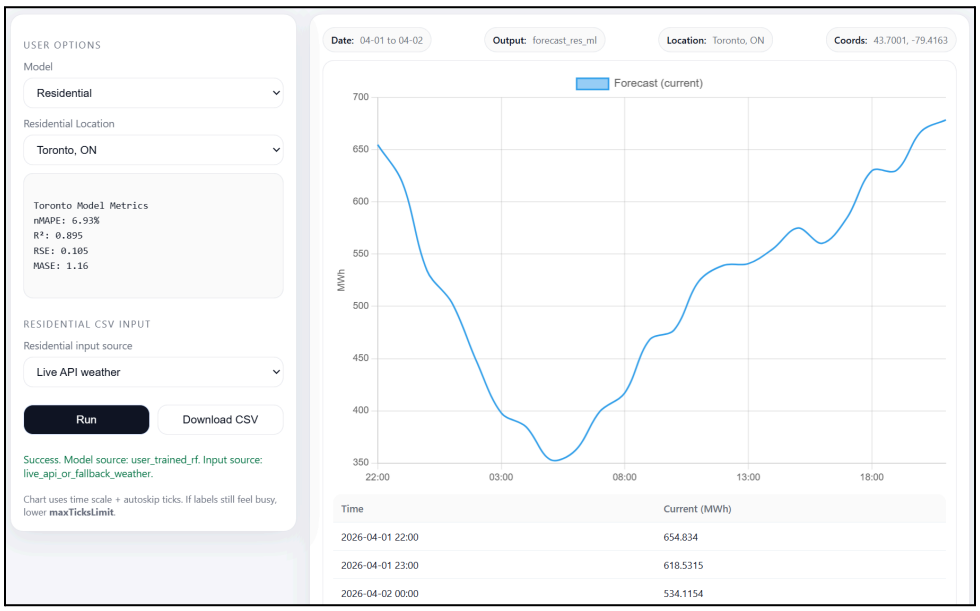


Figure 4. Graphical Website User Interface

Material/Components List

The list summarizing the materials and components used in the development of the project is detailed in **Table 1** below.

Table 1. Material and Components Breakdown

Services and Softwares		Price
Libraries	Keras Library	Free
	Sklearn Library	Free
	Numpy Library	Free
	Pandas Library	Free
Google Colab		Free
PyCharm IDE		Free (Student)
VSCode		Free
Jupyter Notebook		Free
API Services	OpenWeatherMap	Free (Student)
	OpenMeteo	Free
Azure	App Service	51.10 USD/month

Measurement and Testing Procedures

The models were first tuned using base features such as temperature, humidity, time, and dew point to establish a baseline performance. These features capture the main factors that influence electricity demand. The model was then evaluated using metrics such as nMAPE, R^2, RSE, and MASE to determine its accuracy and how well it learned the load patterns. After this, additional features such as HDD, CDD, cyclical time, and 'isWeekend' were gradually added to improve performance. At each step, the model was evaluated again and adjusted, allowing for a systematic tuning process to achieve better forecasting accuracy.

Performance Measurement Results

LSTM Model:

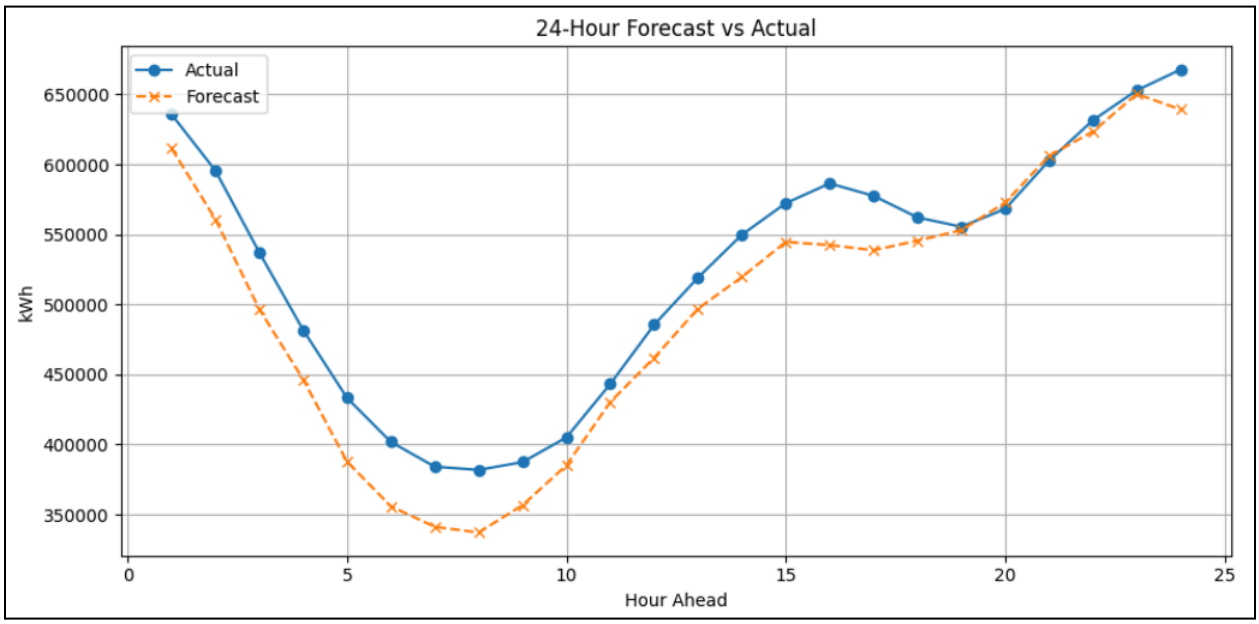


Figure 5. Plot of Toronto Residential Module Forecast vs Actuals

Table 2. Residential Model Performance Metrics

Model	Metrics
Toronto Residential	nMAPE: 6.93% R^2: 0.895 RSE: 0.105 MASE: 1.16 Peak nMAPE: 6.81% Off-Peak nMAPE: 6.66%
Alberta Residential	nMAPE: 3.53% R^2: 0.9812 RSE: 0.0188 MASE: 0.2993
British Columbia Residential	nMAPE: 4.89% R^2: 0.9791 RSE: 0.0209 MASE: 0.2751

CNN Model:

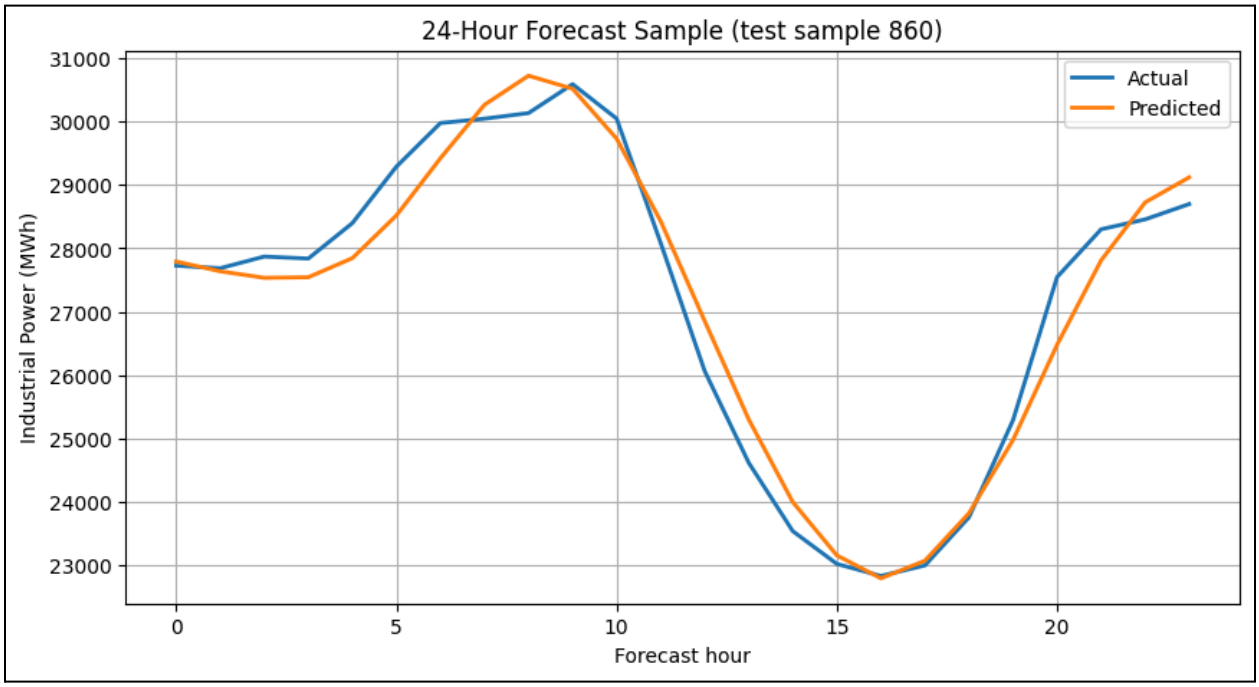


Figure 6. Plot of Ontario Industrial Model Forecast vs Actuals

Table 3. Industrial Model Performance Metrics

Model	Metrics
Ontario Industrial	nMAPE: 4.41% R^2: 0.869 RSE: 0.069 MASE: 1.53
Alberta Industrial	nMAPE: 3.16% R^2: 0.705 RSE: 0.098 MASE: 2.07
BC Industrial	nMAPE: 3.46% R^2: 0.919 RSE: 0.050 MASE: 0.77

Table 4. Commercial Model Performance Metrics

Model	Metrics
Toronto Commercial	nMAPE: 5.84% R^2: 0.861 RSE: 0.092 MASE: 1.12
Alberta Commercial	nMAPE: 4.06% R^2: 0.992 RSE: 0.023 MASE: 0.27
BC Commercial	nMAPE: 1.51% R^2: 0.991 RSE: 0.026 MASE: 0.26

Analysis of Performance

LSTM Model:

The results show that the residential models perform well overall, with low nMAPE values and relatively strong R^2 values. The Alberta Residential model achieves the best performance with a MAPE of 3.53%, showing that the model is able to accurately learn the load patterns. The Toronto Residential and Ontario Residential models also show good performance with consistent predictions with MAPEs of 6.93% and 4.89% respectively. The Ontario model meets the benchmark of around 5%, while the Toronto model performs slightly above this threshold but still demonstrates reasonable accuracy. The Toronto city forecast performs worse than the others, suggesting that residential load is more difficult to predict at the city level compared to provincial. This is because variations such as individual behavior and localized factors have a greater impact on the load, compared to provincial data which is more homogenized and follows a smoother trend.

Peak and off-peak nMAPE are calculated by grouping the forecasted values based on time of day and calculating the normalized sum of absolute errors over each section. This results in an average percentage error that reflects model accuracy during high demand and low demand periods separately by separating specific time sections. The Toronto residential model had peak and off peak MAPE errors of 6.81% and 6.66% respectively. This shows that the model performs similarly across both periods, with only a small improvement during off peak hours.

CNN Model:

The results show that the CNN models perform strongly overall, with low nMAPE values ($\sim \leq$ 5%) and consistently high R^2 scores ($\sim$ 0.9) across most regions. As is displayed in **Table 4**, the Alberta Commercial and BC Commercial models achieve the best performance, with very low error and near-perfect R^2 values, indicating that the CNN is highly effective at capturing load patterns when the data is stable and well-correlated with weather inputs. The Ontario Industrial and Toronto Commercial models also demonstrate solid performance, with relatively low error and strong R^2 values, suggesting reliable predictions in larger, more aggregated systems. However, the Alberta Industrial model results showcased in **Table 3** present a lower R^2 and higher error compared to the others, indicating that industrial load in this region is more difficult to predict, likely due to irregular or less weather-dependent consumption patterns. Overall, commercial models tend to outperform industrial models, and provincial-level forecasts generally perform better than more localized ones, as they benefit from smoother, more aggregated demand trends. These results confirm that the CNN architecture is effective for short-term load forecasting, particularly when applied to consistent and weather-driven datasets.

Conclusions

In conclusion, the completed intelligence-based load forecasting cloud system was delivered at the end of ELE70B reporting period. Historical weather and energy consumption data was collected and utilised to train the intelligence models, and a Python program for API interfacing with real-time public weather data APIs was developed. LSTM models were developed to execute residential load forecasting, and CNN models were created to perform commercial and industrial load forecasting. Finally, all modules were deployed to the Azure cloud environment, and a functional web interface was designed to allow for user access to load forecasting and model training. Furthermore, the $\sim \leq 5\%$ nMAPE results of the project allow for it to be concluded that both the LSTM and CNN machine learning model types can indeed be utilised for the process of load forecasting, and that these models can be implemented using easy-to-use programming languages such as Python.

Some major difficulties encountered include the lack of publicly available, city-specific load data for cities other than Toronto. This limited the Alberta and British Columbia models to provincial-level forecasting, and is also the reason Toronto does not have an industrial forecast, but rather a provincial Ontario one.

Another difficulty included the training time of the model if a user wanted to input their own data. This action could take up to five minutes, which may not be realistic in a real world application, where users expect faster results. Therefore, given more time, the training models could be optimized further to reduce overall training time. Additionally, limited computational resources provided by Microsoft Azure resulted in the website occasionally running slower than average and causing run time errors. This could be improved by upscaling the Azure plan, however the cost to do so would also see a major increase.

Future potential additions to the forecasting system include a larger selection of pre-determined forecasting locations, additional energy sectors, and longer forecasting windows. Additionally, model performance could be adjusted to include more data features and decreased training times.

References

[1] A. McGrath and A. Jonker. “What is Load Forecasting?”, IBM [Online]. Available: https://www.ibm.com/think/topics/load-forecasting [Accessed Nov. 25, 2025].

[2] K. Tarek, “Future Forecasting of Time Series Using LSTM: A Quick Guide for Business Leaders,” Data Science Collective, Medium, 2025.

[3] K. Tarek, “Future Forecasting of Time Series Using LSTM: A Quick Guide for Business Leaders,”Medium, Data Science Collective, 2025. [Online]. Available: https://medium.com/data-science-collective/future-forecasting-of-time-series-using-lstm-a-quick-guide-for-business-leaders-370661c574c9

[4] “Factors Influencing Epoch Count in Machine Learning,” Nebius, 2024. [Online]. Available: https://nebius.com/blog/posts/factors-influencing-epoch-count

[5] M. Hasan *et al.*, “A state-of-the-art comparative review of load forecasting methods: Characteristics, Perspectives, and applications,” *Energy Conversion and Management: X*, vol. 26, p. 100922, Apr. 2025. doi:10.1016/j.ecmx.2025.100922